Mongoose with Express.js

Complete Reference Guide

Beginner · Intermediate · Advanced

Node.js	Express.js	Mongoose	MongoDB

Table of Contents

■ BEGINNER
01 What is Mongoose?
02 Installation & Setup
03 Connecting to MongoDB
04 Defining a Schema
05 Creating a Model
06 CRUD – Create (POST)
07 CRUD – Read (GET)
08 CRUD – Update (PUT)
09 CRUD – Delete (DELETE)

■ INTERMEDIATE
10 Schema Data Types
11 Schema Validation
12 Query Helpers (find, findOne, findById)
13 Filtering & Sorting
14 Populate (References / Joins)
15 Middleware (pre / post hooks)
16 Instance & Static Methods
17 Virtuals

■ ADVANCED
18 Aggregation Pipeline
19 Transactions
20 Indexing for Performance
21 Error Handling & Custom Errors
22 Pagination
23 Soft Delete Pattern
24 Full REST API Example

Beginner

Setup, connection, schemas, models, and basic CRUD operations

01 What is Mongoose?

Mongoose is an ODM (Object Data Modeling) library for MongoDB and Node.js. It provides a schema-based solution to model your application data. Instead of writing raw MongoDB queries, you define schemas and models that give you validation, type casting, query building, and business logic hooks.

02 Installation & Setup

Install Express and Mongoose using npm:

```
# Initialize project
npm init -y

# Install dependencies
npm install express mongoose dotenv
npm install --save-dev nodemon

# package.json scripts
"scripts": {
"start": "node server.js",
"dev": "nodemon server.js"
}
```

03 Connecting to MongoDB

Create a db.js file to handle the MongoDB connection. Always use environment variables for the connection string.

```
// db.js
const mongoose = require('mongoose');

const connectDB = async () => {
try {
const conn = await mongoose.connect(process.env.MONGO_URI);
console.log(`MongoDB Connected: ${conn.connection.host}`);
} catch (error) {
console.error(`Error: ${error.message}`);
process.exit(1);
}
};

module.exports = connectDB;

// server.js
require('dotenv').config();
const express = require('express');
const connectDB = require('./db');

const app = express();
app.use(express.json());

connectDB();

app.listen(5000, () => console.log('Server running on port 5000'));

// .env
MONGO_URI=mongodb://localhost:27017/mydb
PORT=5000
```

04 Defining a Schema

A schema defines the structure of your documents — the fields, types, and constraints.

```
// models/User.js
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
name: {
type: String,
required: [true, 'Name is required'],
trim: true,
},
email: {
type: String,
required: true,
unique: true,
lowercase: true,
},
age: {
type: Number,
min: 0,
max: 120,
},
role: {
type: String,
enum: ['user', 'admin', 'moderator'],
default: 'user',
},
createdAt: {
type: Date,
default: Date.now,
},
});
```

05 Creating a Model

A model is a class built from the schema. It is used to interact with the MongoDB collection.

```
// Continuing in models/User.js

const User = mongoose.model('User', userSchema);
// Mongoose will create a 'users' collection automatically

module.exports = User;
```

06 CRUD – Create (POST)

Use model.create() or new Model().save() to insert documents.

```
// routes/users.js
const express = require('express');
const router = express.Router();
const User = require('../models/User');

// POST /api/users
router.post('/', async (req, res) => {
try {
const user = await User.create(req.body);
// OR: const user = new User(req.body); await user.save();
res.status(201).json({ success: true, data: user });
} catch (error) {
res.status(400).json({ success: false, error: error.message });
}
});

module.exports = router;
```

07 CRUD – Read (GET)

Use .find() to get all, .findById() for a single document.

```
// GET /api/users → all users
router.get('/', async (req, res) => {
try {
const users = await User.find();
res.json({ success: true, count: users.length, data: users });
} catch (error) {
res.status(500).json({ success: false, error: error.message });
}
});

// GET /api/users/:id → single user
router.get('/:id', async (req, res) => {
try {
const user = await User.findById(req.params.id);
if (!user) return res.status(404).json({ message: 'User not found' });
res.json({ success: true, data: user });
} catch (error) {
res.status(500).json({ success: false, error: error.message });
}
});
```

08 CRUD – Update (PUT)

Use .findByIdAndUpdate() with {new: true} to return the updated document.

```
// PUT /api/users/:id
router.put('/:id', async (req, res) => {
try {
const user = await User.findByIdAndUpdate(
req.params.id,
req.body,
{ new: true, runValidators: true }
);
if (!user) return res.status(404).json({ message: 'Not found' });
res.json({ success: true, data: user });
} catch (error) {
res.status(400).json({ success: false, error: error.message });
}
});
```

09 CRUD – Delete (DELETE)

Use .findByIdAndDelete() to remove a document.

```
// DELETE /api/users/:id
router.delete('/:id', async (req, res) => {
try {
const user = await User.findByIdAndDelete(req.params.id);
if (!user) return res.status(404).json({ message: 'Not found' });
res.json({ success: true, message: 'User deleted' });
} catch (error) {
res.status(500).json({ success: false, error: error.message });
}
});
```

Intermediate

Validation, querying, populate (joins), hooks, and virtual fields

10 Schema Data Types

Mongoose supports all common data types. Here is a reference schema showing all types:

```
const schema = new mongoose.Schema({
name: String,
age: Number,
isActive: Boolean,
birthday: Date,
scores: [Number], // array of numbers
tags: [String], // array of strings
address: { // nested object
street: String,
city: String,
zip: String,
},
avatar: Buffer, // binary data
meta: mongoose.Schema.Types.Mixed, // any type
userId: mongoose.Schema.Types.ObjectId,
bigNum: BigInt,
});
```

11 Schema Validation

Mongoose has built-in validators and supports custom validators.

```
const productSchema = new mongoose.Schema({
name: {
type: String,
required: [true, 'Product name required'],
minlength: [3, 'Min 3 characters'],
maxlength: 100,
},
price: {
type: Number,
required: true,
min: [0, 'Price cannot be negative'],
},
email: {
type: String,
match: [/^\S+@\S+\.\S+$/, 'Invalid email'],
},
category: {
type: String,
enum: { values: ['electronics','clothing','food'],
message: '{VALUE} is not a valid category' },
},
// Custom validator
discount: {
type: Number,
validate: {
validator: (v) => v >= 0 && v <= 100,
message: 'Discount must be between 0 and 100',
},
},
});
```

12 Query Helpers

Mongoose provides many built-in query helpers for reading data.

```
// find all
const users = await User.find();

// find with filter
const admins = await User.find({ role: 'admin' });

// findOne – first matching document
const user = await User.findOne({ email: 'a@b.com' });

// findById – by MongoDB _id
const user = await User.findById('64f2a...');

// select specific fields (projection)
const users = await User.find().select('name email -_id');

// limit and skip
const users = await User.find().limit(10).skip(20);

// count
const total = await User.countDocuments({ role: 'user' });
```

13 Filtering & Sorting

Use MongoDB query operators inside find() for advanced filtering.

```
// Comparison operators
await User.find({ age: { $gte: 18, $lte: 60 } });
await User.find({ role: { $in: ['admin', 'moderator'] } });
await User.find({ name: { $ne: 'John' } });

// Logical operators
await User.find({
$or: [{ age: { $lt: 18 } }, { role: 'admin' }]
});

// Text search (needs text index)
await User.find({ $text: { $search: 'nodejs' } });

// Sorting (1 = asc, -1 = desc)
await User.find().sort({ createdAt: -1 });
await User.find().sort({ name: 1, age: -1 });

// Regex search
await User.find({ name: { $regex: /john/i } });
```

14 Populate (References / Joins)

Use ObjectId references between schemas and .populate() to join documents.

```
// Post schema references User
const postSchema = new mongoose.Schema({
title: String,
body: String,
author: {
type: mongoose.Schema.Types.ObjectId,
ref: 'User', // 'User' is the model name
required: true,
},
tags: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Tag' }],
});

// Populate author when reading posts
const posts = await Post
.find()
.populate('author', 'name email') // select only name & email
.populate('tags', 'name');

// Nested populate
await Post.find().populate({
path: 'author',
populate: { path: 'department' } // author's department
});
```

15 Middleware (pre / post hooks)

Hooks run before or after Mongoose operations. Useful for hashing passwords, logging, etc.

```
const bcrypt = require('bcryptjs');

userSchema.pre('save', async function(next) {
// 'this' refers to the document being saved
if (!this.isModified('password')) return next();
this.password = await bcrypt.hash(this.password, 12);
next();
});

// post hook – runs after the operation
userSchema.post('findOneAndDelete', async function(doc) {
if (doc) {
// delete all posts by this user
await Post.deleteMany({ author: doc._id });
}
});

// pre-find hook (e.g. filter out inactive users globally)
userSchema.pre(/^find/, function(next) {
this.find({ isActive: { $ne: false } });
next();
});
```

16 Instance & Static Methods

Add reusable methods directly to your model.

```
// Instance method – called on a document
userSchema.methods.comparePassword = async function(candidatePass) {
return bcrypt.compare(candidatePass, this.password);
};

// Usage:
const user = await User.findOne({ email });
const isMatch = await user.comparePassword(req.body.password);

// Static method – called on the Model
userSchema.statics.findByEmail = function(email) {
return this.findOne({ email: email.toLowerCase() });
};

// Usage:
const user = await User.findByEmail('test@example.com');
```

17 Virtuals

Virtuals are properties not stored in MongoDB. Great for computed fields.

```
userSchema.virtual('fullName').get(function() {
return `${this.firstName} ${this.lastName}`;
});

// Include virtuals in JSON output
const userSchema = new mongoose.Schema({ ... }, {
toJSON: { virtuals: true },
toObject: { virtuals: true },
});

// Getter + Setter virtual
userSchema.virtual('fullName')
.get(function() { return `${this.first} ${this.last}`; })
.set(function(v) {
const parts = v.split(' ');
this.first = parts[0];
this.last = parts[1];
});
```

Advanced

Aggregation, transactions, indexing, error handling, pagination & full API example

18 Aggregation Pipeline

Aggregation processes data records and returns computed results. Powerful for analytics.

```
// Total revenue per category
const result = await Order.aggregate([
{ $match: { status: 'completed' } }, // filter
{ $group: {
_id: '$category',
total: { $sum: '$amount' },
count: { $sum: 1 },
avgAmt: { $avg: '$amount' },
}},
{ $sort: { total: -1 } }, // sort desc
{ $limit: 5 }, // top 5
{ $project: { _id: 0, category: '$_id', total: 1, count: 1 } },
]);

// $lookup (like SQL JOIN)
const result = await Post.aggregate([
{ $lookup: {
from: 'users',
localField: 'author',
foreignField: '_id',
as: 'authorInfo',
}},
{ $unwind: '$authorInfo' },
]);
```

19 Transactions

Use sessions and transactions for multi-document atomic operations (requires MongoDB replica set).

```
const session = await mongoose.startSession();

try {
session.startTransaction();

// Deduct balance from sender
await Account.findByIdAndUpdate(
senderId,
{ $inc: { balance: -amount } },
{ session }
);

// Add balance to receiver
await Account.findByIdAndUpdate(
receiverId,
{ $inc: { balance: +amount } },
{ session }
);

await session.commitTransaction();
res.json({ success: true });

} catch (error) {
await session.abortTransaction();
res.status(500).json({ error: error.message });
} finally {
session.endSession();
}
```

20 Indexing for Performance

Indexes speed up queries dramatically. Always index fields you filter or sort on frequently.

```
// Single field index (in schema)
userSchema.index({ email: 1 }); // ascending
userSchema.index({ createdAt: -1 }); // descending

// Compound index
userSchema.index({ lastName: 1, firstName: 1 });

// Unique index
userSchema.index({ email: 1 }, { unique: true });

// Text index for full-text search
postSchema.index({ title: 'text', body: 'text' });
// Usage: Post.find({ $text: { $search: 'mongoose' } })

// TTL index – auto-delete documents after N seconds
sessionSchema.index({ createdAt: 1 }, { expireAfterSeconds: 3600 });

// Shorthand in field definition
email: { type: String, index: true, unique: true }
```

21 Error Handling & Custom Errors

Centralized error handling with custom error classes for clean API responses.

```
// utils/AppError.js
class AppError extends Error {
constructor(message, statusCode) {
super(message);
this.statusCode = statusCode;
this.isOperational = true;
}
}
module.exports = AppError;

// middleware/errorHandler.js
const errorHandler = (err, req, res, next) => {
let error = { ...err, message: err.message };

// Mongoose bad ObjectId
if (err.name === 'CastError')
error = new AppError('Resource not found', 404);

// Mongoose duplicate key
if (err.code === 11000) {
const field = Object.keys(err.keyValue)[0];
error = new AppError(`Duplicate value for ${field}`, 400);
}

// Mongoose validation error
if (err.name === 'ValidationError') {
const msg = Object.values(err.errors).map(e => e.message);
error = new AppError(msg.join('. '), 400);
}

res.status(error.statusCode || 500).json({
success: false,
error: error.message || 'Server Error',
});
};
module.exports = errorHandler;

// In server.js (must be LAST middleware)
app.use(errorHandler);
```

22 Pagination

Implement page-based or cursor-based pagination for large datasets.

```
// GET /api/posts?page=2&limit=10
router.get('/', async (req, res) => {
const page = parseInt(req.query.page) || 1;
const limit = parseInt(req.query.limit) || 10;
const skip = (page - 1) * limit;

const [posts, total] = await Promise.all([
Post.find().skip(skip).limit(limit).sort({ createdAt: -1 }),
Post.countDocuments(),
]);

res.json({
success: true,
data: posts,
pagination: {
page,
limit,
total,
pages: Math.ceil(total / limit),
hasNext: page * limit < total,
hasPrev: page > 1,
},
});
});
```

23 Soft Delete Pattern

Instead of permanently deleting, mark documents as deleted. Useful for data recovery.

```
// Add deletedAt to schema
userSchema.add({
deletedAt: { type: Date, default: null },
isDeleted: { type: Boolean, default: false },
});

// Global filter – hide deleted docs in all queries
userSchema.pre(/^find/, function(next) {
this.where({ isDeleted: false });
next();
});

// Soft delete method
userSchema.methods.softDelete = async function() {
this.isDeleted = true;
this.deletedAt = new Date();
return this.save();
};

// Route usage
router.delete('/:id', async (req, res) => {
const user = await User.findById(req.params.id);
await user.softDelete();
res.json({ success: true, message: 'User soft deleted' });
});
```

24 Full REST API Example

A complete, production-ready file structure and API for a Blog system.

```
// ■■ Project Structure ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
project/
■■■ models/
■ ■■■ User.js
■ ■■■ Post.js
■■■ routes/
■ ■■■ users.js
■ ■■■ posts.js
■■■ middleware/
■ ■■■ errorHandler.js
■■■ utils/
■ ■■■ AppError.js
■■■ db.js
■■■ server.js
■■■ .env
```

```
// ■■ models/Post.js ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
const postSchema = new mongoose.Schema({
title: { type: String, required: true, trim: true },
content: { type: String, required: true },
author: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
tags: [String],
likes: { type: Number, default: 0 },
views: { type: Number, default: 0 },
}, { timestamps: true });

postSchema.index({ title: 'text', content: 'text' });
module.exports = mongoose.model('Post', postSchema);
```

```
// ■■ routes/posts.js (full CRUD) ■■■■■■■■■■■■■■■■■■■■■■
const router = express.Router();

// GET /api/posts?page=1&limit=10&search=node
router.get('/', async (req, res, next) => {
try {
const { page=1, limit=10, search, tag } = req.query;
const filter = {};
if (search) filter.$text = { $search: search };
if (tag) filter.tags = tag;

const posts = await Post.find(filter)
.populate('author', 'name email')
.sort({ createdAt: -1 })
.skip((page - 1) * limit)
.limit(Number(limit));

const total = await Post.countDocuments(filter);
res.json({ success: true, data: posts,
pagination: { page: +page, total, pages: Math.ceil(total/limit) }
});
} catch (err) { next(err); }
});

// POST /api/posts
router.post('/', async (req, res, next) => {
try {
const post = await Post.create({ ...req.body, author: req.user._id });
res.status(201).json({ success: true, data: post });
} catch (err) { next(err); }
});

// PATCH /api/posts/:id/like
router.patch('/:id/like', async (req, res, next) => {
try {
const post = await Post.findByIdAndUpdate(
req.params.id,
{ $inc: { likes: 1 } },
{ new: true }
);
res.json({ success: true, data: post });
} catch (err) { next(err); }
});

module.exports = router;
```

Quick Reference — Common Methods

Method	Description

`Model.create(data)`	Insert one or more documents
`Model.find(filter)`	Get all matching documents
`Model.findOne(filter)`	Get first matching document
`Model.findById(id)`	Find by MongoDB _id
`Model.findByIdAndUpdate(id,upd)`	Find, update and return document
`Model.findByIdAndDelete(id)`	Find and delete document
`Model.countDocuments(filter)`	Count matching documents
`Model.aggregate([...])`	Run aggregation pipeline
`.populate('field')`	Replace ObjectId with full document
`.select('a b -c')`	Include/exclude fields
`.sort({ field: -1 })`	Sort results
`.limit(n).skip(n)`	Pagination
`.lean()`	Return plain JS object (faster, no methods)